4080. Design Bus Seat Booking System

Solved

Medium Hint

You are asked to design a seat booking system for a bus.

The bus has `n` rows. Each row has four seats labeled `"A"`, `"B"`, `"C"`, and `"D"`.

Seats `"A"` and `"D"` are **window** seats; seats `"B"` and `"C"` are **middle** seats. In each row, seat `"A"` is **adjacent** to seat `"B"`, and seat `"D"` is **adjacent** to seat `"C"`.

Booking rules:

- Booking a **middle** seat (`"B"` or `"C"`) always takes 1 second.
- Booking a **window** seat (`"A"` or `"D"`) takes 1 second if its **adjacent middle** seat is **not** booked, or 3 seconds (an extra delay of 2 seconds) if its **adjacent middle** seat is **already** booked.

A seat is identified by a string such as `"A1"` or `"D20"` (one letter from `A` to `D` followed by its row number).

Implement the `BusBooking` class:

- `BusBooking(int n)`: Initializes the system with `n` **empty** rows.
- `void toggle(string seat)`: Books `seat` if it is free, or frees it if it is already booked.
- `int getTotalTime()`: Returns the total time (in seconds) taken to book all **currently** booked seats, where the seats are booked in the order their `toggle` calls were made. If no seats are currently booked, return 0.
- `int getMinTime()`: Returns the **smallest** possible total time (in seconds) to book all **currently** booked seats, over every possible order of booking them. If no seats are currently booked, return 0.

Note: The time cost of a seat is fixed at the moment it is booked, based on the seats booked at that time. Booking or freeing other seats afterward does not change the cost of a seat that is already booked.

Example 1:

> **Input:**
> ["BusBooking", "toggle", "toggle", "getTotalTime", "getMinTime", "toggle", "toggle", "getTotalTime", "getMinTime"]
> [[20], ["B3"], ["A3"], [], [], ["C3"], ["D3"], [], []]
>
> **Output:**
> [null, null, null, 4, 2, null, null, 8, 4]
>
> **Explanation**
>
> BusBooking busBooking = new BusBooking(20); // start with 20 empty rows
> busBooking.toggle("B3"); // book `"B3"`, a middle seat, which costs 1 second. Total time is now 1.
> busBooking.toggle("A3"); // book `"A3"`, a window seat. Its adjacent middle `"B3"` is already booked, so it costs 3 seconds. Total time is now `1 + 3 = 4`.
> busBooking.getTotalTime(); // return 4, since in toggle order `"B3"` costs 1 and `"A3"` costs 3.
> busBooking.getMinTime(); // return 2. Booking `"A3"` before `"B3"` avoids the delay, so 2 seats takes 2 second to book.
> busBooking.toggle("C3"); // book `"C3"`, a middle seat, which costs 1 second. Total time is now `1 + 4 = 5`.
> busBooking.toggle("D3"); // book `"D3"`, a window seat. Its adjacent middle `"C3"` is already booked, so it costs 3 seconds. Total time is now `5 + 3 = 8`.
> busBooking.getTotalTime(); // return 8, since in toggle order the seat costs are 1, 3, 1, 3.
> busBooking.getMinTime(); // return 4. With the best possible booking order, no delay is added, so the 4 booked seats take `4 * 1 = 4` seconds.

Example 2:

> **Input:**
> ["BusBooking", "toggle", "toggle", "getTotalTime", "toggle", "toggle", "toggle", "getTotalTime", "getMinTime"]
> [[10], ["C7"], ["D7"], [], ["D7"], ["C7"], ["D7"], [], []]
>
> **Output:**
> [null, null, null, 4, null, null, null, 1, 1]
>
> **Explanation**
>
> BusBooking busBooking = new BusBooking(10); // start with 10 empty rows
> busBooking.toggle("C7"); // book `"C7"`, a middle seat, which costs 1 second. Total time is now 1.
> busBooking.toggle("D7"); // book `"D7"`, a window seat. Its adjacent middle `"C7"` is already booked, so it costs 3 seconds. Total time is now `1 + 3 = 4`.
> busBooking.getTotalTime(); // return 4, since in toggle order `"C7"` costs 1 and `"D7"` costs 3.
> busBooking.toggle("D7"); // `"D7"` is already booked, so free it. This removes its cost of 3. Total time is now `4 - 3 = 1`.
> busBooking.toggle("C7"); // `"C7"` is already booked, so free it. This removes its cost of 1. Total time is now `1 - 1 = 0`, and row 7 is empty.
> busBooking.toggle("D7"); // book `"D7"` again, a window seat. Its adjacent middle `"C7"` is not booked this time, so it costs only 1 second. Total time is now `0 + 1 = 1`.
> busBooking.getTotalTime(); // return 1, since only `"D7"` is booked and it cost 1 this time.
> busBooking.getMinTime(); // return 1. With the best possible booking order, the 1 booked seat takes `1 * 1 = 1` second.

Constraints:

- `1 <= n <= 10^5`
- `seat[0]` is one of `'A'`, `'B'`, `'C'`, `'D'`.
- The remaining characters of `seat` form an integer `row` with `1 <= row <= n`.
- The total number of calls to `toggle`, `getTotalTime`, and `getMinTime` does not exceed `10^5`.

Seen this question in a real interview before? 1/6

Yes No

Accepted 10,101/22.4K | Acceptance Rate 45.1%

Topics

Hint 1

Hint 2

Hint 3

Discussion (10)

Copyright © 2026 LeetCode. All rights reserved.